

Mini ERP + CRM Operations Portal

Final Submission Document

Project: Fundsroom | Full Stack Developer Case Study

1. Submission Checklist

Item	Submission	Status
1	GitHub repository	Ready
2	Live frontend URL	Ready
3	Live backend API URL	Ready
4	Test login credentials for all roles	Ready
5	Postman collection / API documentation	Included
6	README with setup & deployment instructions	Included in repository
7	Architecture explanation	Included
8	Known limitations / incomplete parts	Included

2. Live Project & Repository

GitHub	https://github.com/Satvik0609/Fundsroom
Live Frontend	https://fundsroom-ivory.vercel.app
Live Backend API	https://mini-erp-crm-api-h0d8.onrender.com
Backend Health Check	https://mini-erp-crm-api-h0d8.onrender.com/health

3. Test Login Credentials

Role	Email	Password
Admin	admin@erp.local	Pass@123
Sales	sales@erp.local	Pass@123
Warehouse	warehouse@erp.local	Pass@123
Accounts	accounts@erp.local	Pass@123

Note: These are demo credentials seeded into the production database for evaluation. Production secrets such as DATABASE_URL and JWT_SECRET are not included in this document.

4. Project Overview

Fundsroom is a full-stack Mini ERP + CRM Operations Portal designed for a wholesale/distribution business. It supports internal Sales, Warehouse, Accounts, and Admin users across customer management, product inventory, stock movements, and sales challans. The repository README describes the project as a production-quality case-study implementation demonstrating REST APIs, PostgreSQL database design, JWT authentication, role-based authorization, and a React admin UI.

5. Core Modules

Module	Implemented capabilities
Authentication & Roles	JWT authentication, bcrypt password hashing, role-based access for Admin, Sales, Warehouse and Accounts.
Customer CRM	Customer CRUD, search, pagination, customer details, follow-up history and notes.
Product & Inventory	Product catalog, SKU, pricing, current stock, minimum-stock indicators and warehouse/location information.
Stock Movements	IN/OUT movement tracking with quantity, reason, creator and timestamp.
Sales Challans	Create draft challans, add multiple products/quantities, automatic challan numbering, confirm/cancel.
Business Logic	Atomic confirmation transaction, stock validation, no negative stock, product snapshot data on confirmation.

6. Architecture

The application follows a three-layer full-stack architecture. The React/Vite frontend communicates with the Node.js/TypeScript/Express REST API using JSON and JWT authentication. The backend uses Prisma ORM to access PostgreSQL. The deployed frontend runs on Vercel, the backend runs on Render, and the production PostgreSQL database is hosted on Neon.

Deployment flow: GitHub → Vercel (Frontend) and Render (Backend) → Neon PostgreSQL (Database).

Application flow: React SPA ↔ Express REST API ↔ Prisma ORM ↔ PostgreSQL.

7. Technology Stack

Layer	Technology
Frontend	React, TypeScript, Vite, HTML/CSS
Backend	Node.js, TypeScript, Express.js
Database	PostgreSQL
ORM	Prisma
Authentication	JWT, bcryptjs
Validation	Zod
Testing	Jest, Supertest
Deployment	Vercel, Render, Neon
API Style	REST / JSON

8. API Documentation

The repository contains a Postman collection at `postman/Mini-ERP-CRM.postman_collection.json`. The collection is intended for API testing and includes negative tests for validation and insufficient stock. The following endpoints are documented in the project README.

Method	Endpoint	Access / Purpose
POST	<code>/auth/login</code>	Public — login
GET	<code>/auth/me</code>	All roles — current user
GET	<code>/health</code>	Public — health check
GET	<code>/dashboard</code>	All roles — dashboard statistics
GET / POST	<code>/customers</code>	Sales+ — list/create customers
GET / PUT	<code>/customers/:id</code>	Sales+ — get/update customer
POST	<code>/customers/:id/followups</code>	Sales+ — add follow-up
GET / POST	<code>/products</code>	Warehouse+ — list/create products
PUT	<code>/products/:id</code>	Warehouse+ — update product
GET / POST	<code>/products/:id/movements</code>	Warehouse+ — stock movements
GET	<code>/products/movements/all</code>	All roles — all movements
GET / POST	<code>/challans</code>	Sales+ — list/create challans
GET	<code>/challans/:id</code>	Sales+ — get challan
PUT	<code>/challans/:id</code>	Sales+ — update draft challan
POST	<code>/challans/:id/confirm</code>	Sales+ — confirm challan
POST	<code>/challans/:id/cancel</code>	Sales+ — cancel draft challan

9. Critical Business Logic — Sales Challan

Draft challans do not affect stock. During confirmation, the backend starts a database transaction, loads all challan items, validates available stock for every product, and rejects the operation with HTTP 409 if any item would cause insufficient stock. In that failure case, stock and stock movements remain unchanged and the challan stays in DRAFT. If validation succeeds, stock is reduced, OUT movements are created, the challan is marked CONFIRMED, and the transaction is committed.

State rules documented in the repository: DRAFT → CONFIRMED; DRAFT → CANCELLED; confirmed/cancelled states cannot be incorrectly transitioned back into draft or reconfirmed.

10. Local Setup

Prerequisites: Node.js 18+, PostgreSQL 14+, and npm. The repository README provides the following setup flow:

Backend

```
cd backend
cp .env.example .env
# Configure DATABASE_URL and JWT_SECRET
npm install
npx prisma generate
```

```
npx prisma migrate dev --name init
npm run db:seed
npm run dev
```

Frontend

```
cd frontend
cp .env.example .env
npm install
npm run dev
```

11. Environment Variables

Environment-specific configuration is used for backend and frontend. Production secrets are managed in the hosting platforms and are intentionally not included in this document.

Variable	Purpose
DATABASE_URL	PostgreSQL/Neon connection string
JWT_SECRET	JWT signing secret
CORS_ORIGIN	Allowed deployed frontend origin
PORT	Backend service port
NODE_ENV	Runtime environment
VITE_API_URL	Frontend API base URL

12. Deployment

Database: Neon PostgreSQL. The production database was seeded with demo users, customers, products, stock movements and a draft challan.

Backend: Render. The deployed API is available at the live backend URL listed above. The health endpoint returns a successful status response.

Frontend: Vercel. The frontend is configured with the Render API URL through VITE_API_URL.

CORS: The backend CORS_ORIGIN is configured to allow the deployed Vercel frontend origin.

13. Assumptions

- Indian mobile number format: 10 digits starting with 6-9.
- INR is used for currency formatting.
- GST number validation follows the standard 15-character format.
- Only draft challans can be cancelled; confirmed challans do not have a stock-reversal workflow.
- Challan numbers follow the SC-YYYY-NNNNNN format.

14. Known Limitations / Incomplete Parts

- PDF invoice export is not implemented.
- Email notifications are not implemented.
- Multi-warehouse support is not implemented.
- Purchase order module is not implemented.
- The application currently uses a single currency (INR).

15. Future Improvements

- PDF challan/invoice export.
- Docker Compose one-command setup.
- GitHub Actions CI pipeline.
- Purchase orders and supplier management.
- Advanced reporting and analytics.
- Audit logging for all changes.

16. Final Evaluator Access

For evaluation, open the live frontend URL first and use one of the demo accounts in Section 3. The backend health endpoint can be used independently to verify API availability. The GitHub repository contains the source code, README, deployment documentation, Postman collection, database setup documentation, and supporting configuration.

Submission package: GitHub repository + live frontend + live backend + four role credentials + Postman/API documentation + README + architecture + limitations.